head_specialization

October 6, 2026

1 Especialización funcional entre cabezas de un bloque — vectores de daño por clase (2026-08-30)

Bloque B del capítulo (*circuit study*), extensión de `notebooks/22082026_ablation/ablation.ipynb`. La ablación de ese notebook mide *cuánto* cae la accuracy global al apagar cada cabeza. Acá se mira *a qué clases* le pega cada cabeza: se ablaciona una cabeza a la vez, se calcula la caída de accuracy **por clase** (vector de dim 10), y se comparan esos "vectores de daño" entre cabezas del mismo bloque.

- Si dos cabezas tienen vectores de daño muy **parecidos** (coseno / Pearson altos) -> hacen un trabajo redundante: apagar una o la otra rompe las mismas clases.
- Si son **distintos / ortogonales** -> están especializadas en clases diferentes.

La hipótesis del capítulo es que el modelo penalizado ("interpretable") tiene cabezas con vectores de daño **menos correlacionados** que el baseline (menos información compartida entre cabezas).

Ambos modelos en una corrida. La celda de configuración elige `BLOCK_IDX` y carga `MODELS = {"baseline": ..., "interpretable": ...}`; cada tarea calcula y grafica para los dos, y cada imagen se guarda con el nombre del modelo (y el bloque) en el archivo, sin pisarse entre sí. El nombre exacto con el que se guardó cada imagen se imprime debajo de la gráfica.

Sample de referencia: las primeras `N_REF=1000` imágenes del test de AwA2, con sus labels.

1.1 0. Configuración — carga ambos modelos

[1]:
```
# ======================================================================
#  CONFIGURACIÓN
# ======================================================================
BLOCK_IDX = 4   # bloque a analizar, 1-indexed. Internamente se pasa a␣
 ↪0-indexed.
N_REF     = 1000
# ======================================================================

import json
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
```

```
import pandas as pd
import torch

# Misma convención que el resto de notebooks/: vive 2 niveles bajo la raíz del proyecto.
project_root = Path.cwd().resolve().parents[1]
sys.path.append(str(project_root))

from modules.model import ViTForClassfication
from modules.datasets import AwA2Dataset
from modules.figures import save_fig

torch.manual_seed(0)

EXPERIMENTS_DIR = project_root / "experimentation" / "experiments"
OUTPUT_DIR = EXPERIMENTS_DIR / "30082026_head_specialization"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

MODELS = {
    "baseline": "vit-with-15-epochs-AwA2",
    "interpretable": "vit-with-15-epochs-interpretable-AwA2",
}

print(f"BLOCK_IDX  : {BLOCK_IDX}  (1-indexed)")
print(f"OUTPUT_DIR : {OUTPUT_DIR.relative_to(project_root)}")
print(f"N_REF      : {N_REF}")
```

```
BLOCK_IDX  : 4  (1-indexed)
OUTPUT_DIR : experimentation/experiments/30082026_head_specialization
N_REF      : 1000
```

1.2 1. Cargar los dos modelos

El `assert` sobre `use_faster_attention` protege la ablación por cabeza (asume `block.attention.heads` como submódulos separados). Se convierte `BLOCK_IDX` (1-indexed) al índice 0-indexed `block_idx0` que usan las funciones de ablación.

[2]:
```
def load_model(name):
    exp_dir = EXPERIMENTS_DIR / name
    with open(exp_dir / "config.json") as f:
        config = json.load(f)
    with open(exp_dir / "metrics.json") as f:
        metrics = json.load(f)
    assert not config["use_faster_attention"], (
        "Este notebook asume block.attention.heads (cabezas separadas); "
        "con use_faster_attention=True habría que adaptar la ablación por cabeza."
    )
```

```
    model = ViTForClassfication(config)
    model.load_state_dict(torch.load(exp_dir / "model_final.pt",␣
 ↪map_location="cpu"))
    model.eval()
    return model, config, metrics


models = {key: load_model(name) for key, name in MODELS.items()}

config = models["baseline"][1]
assert config == models["interpretable"][1], "Los dos modelos deben tener la␣
 ↪misma arquitectura"

n_blocks    = len(models["baseline"][0].encoder.blocks)
n_heads     = config["num_attention_heads"]
num_classes = config["num_classes"]

assert 1 <= BLOCK_IDX <= n_blocks, f"BLOCK_IDX debe estar en 1..{n_blocks}␣
 ↪(1-indexed); recibí {BLOCK_IDX}"
block_idx0 = BLOCK_IDX - 1
head_labels = [f"head {h + 1}" for h in range(n_heads)]

for key, (_, _, metrics) in models.items():
    print(f"{key:>13} ({MODELS[key]}): accuracy de entrenamiento =␣
 ↪{metrics['accuracy']:.4f}")
print(f"bloques={n_blocks} | cabezas/bloque={n_heads} | clases={num_classes}")
print(f"Bloque a analizar: block {BLOCK_IDX}  (índice interno {block_idx0})  ->␣
 ↪ {n_heads} cabezas")
```

```
     baseline (vit-with-15-epochs-AwA2): accuracy de entrenamiento = 0.3091
interpretable (vit-with-15-epochs-interpretable-AwA2): accuracy de entrenamiento
= 0.3143
bloques=4 | cabezas/bloque=4 | clases=50
Bloque a analizar: block 4  (índice interno 3)  ->  4 cabezas
```

1.3 2. Sample de referencia (n = 1000, con labels)

Primeras `N_REF` imágenes del test de AwA2 y sus labels de clase. `AwA2Dataset` usa el cache local ya construido en `data/awa2/`, así que no vuelve a descargar nada.

[3]:
```
dataset = AwA2Dataset(image_size=32, root=str(project_root / "data" / "awa2"))
_, testloader, classes_tuple = dataset.get_loaders(batch_size=256)
classes = list(classes_tuple)

ref_images, ref_labels = dataset.get_samples_from_indices(range(N_REF),␣
 ↪device="cpu", set="test")
_counts = torch.bincount(ref_labels, minlength=num_classes).tolist()
```

```
print(f"Sample de referencia: {tuple(ref_images.shape)}  (índices 0..{N_REF -␣
 ↪1} del test)")
print("imágenes por clase:", {c: n for c, n in zip(classes, _counts)})
assert any(n > 0 for n in _counts), "Ninguna clase aparece en el sample; subir␣
 ↪N_REF."
```

```
Sample de referencia: (1000, 3, 32, 32)  (índices 0..999 del test)
imágenes por clase: {'antelope': 209, 'grizzly bear': 0, 'killer whale': 0,
'beaver': 39, 'dalmatian': 0, 'persian cat': 0, 'horse': 0, 'german shepherd':
0, 'blue whale': 35, 'siamese cat': 0, 'skunk': 0, 'mole': 0, 'tiger': 0,
'hippopotamus': 0, 'leopard': 0, 'moose': 0, 'spider monkey': 0, 'humpback
whale': 0, 'elephant': 0, 'gorilla': 0, 'ox': 0, 'fox': 0, 'sheep': 0, 'seal':
0, 'chimpanzee': 146, 'hamster': 0, 'squirrel': 0, 'rhinoceros': 0, 'rabbit': 0,
'bat': 77, 'giraffe': 0, 'wolf': 0, 'chihuahua': 113, 'rat': 0, 'weasel': 0,
'otter': 0, 'buffalo': 179, 'zebra': 0, 'giant panda': 0, 'deer': 0, 'bobcat':
126, 'pig': 0, 'lion': 0, 'mouse': 0, 'polar bear': 0, 'collie': 76, 'walrus':
0, 'raccoon': 0, 'cow': 0, 'dolphin': 0}
```

1.4 3. Funciones: ablación por cabeza + accuracy por clase

`forward_with_head_ablated` es **la misma función de `notebooks/22082026_ablation/ablation.ipynb`** (copiada tal cual): reconstruye la atención del bloque `ablate_layer_idx` a mano, corre cada cabeza por separado y pone en **cero** la salida de `ablate_head_idx` antes de concatenar y proyectar; el resto del modelo queda intacto. Con `ablate_layer_idx=None` no toca nada (se usa para el sanity check).

`per_class_accuracy` desglosa la accuracy en los 10 valores por clase.

[4]:
```
@torch.no_grad()
def forward_with_head_ablated(model, x, ablate_layer_idx=None,␣
 ↪ablate_head_idx=None):
    h = model.embedding(x)
    for i, block in enumerate(model.encoder.blocks):
        if i == ablate_layer_idx:
            attn_in = block.layernorm_1(h)
            head_outputs = []
            for hidx, head in enumerate(block.attention.heads):
                out, _ = head(attn_in)
                if hidx == ablate_head_idx:
                    out = torch.zeros_like(out)   # <- ablación: salida de esa␣
 ↪cabeza a cero
                head_outputs.append(out)
            concat = torch.cat(head_outputs, dim=-1)
            attn_out = block.attention.output_projection(concat)
            attn_out = block.attention.output_dropout(attn_out)
            h = h + attn_out
            mlp_out = block.mlp(block.layernorm_2(h))
            h = h + mlp_out
```

```
        else:
            h, _ = block(h, output_attentions=False)
    return model.classifier(h[:, 0, :])


@torch.no_grad()
def accuracy_of(logits, labels):
    return (logits.argmax(dim=-1) == labels).float().mean().item()


@torch.no_grad()
def per_class_accuracy(logits, labels, num_classes):
    """Vector de dim num_classes: accuracy dentro de cada clase (correctos_c / total_c)."""
    preds = logits.argmax(dim=-1)
    accs = np.full(num_classes, np.nan)
    for c in range(num_classes):
        mask = labels == c
        n_c = int(mask.sum())
        if n_c > 0:
            accs[c] = (preds[mask] == labels[mask]).float().mean().item()
    return accs


# Sanity check: sin ablación, la reimplementación manual debe coincidir EXACTO con el forward
# normal, para cada modelo. También queda cacheado direct_logits por modelo, reutilizado abajo.
direct_logits = {}
for key, (model, _, _) in models.items():
    with torch.no_grad():
        direct_logits[key], _ = model(ref_images, output_attentions=False)
    head_fn_logits = forward_with_head_ablated(model, ref_images, ablate_layer_idx=None, ablate_head_idx=None)
    _diff = (direct_logits[key] - head_fn_logits).abs().max().item()
    assert _diff < 1e-5, f"{key}: sin ablación la función debe reproducir el forward normal"
    print(f"[{key}] diferencia máxima (forward_with_head_ablated, sin ablar) vs forward normal: {_diff:.2e}")
print("OK - la reimplementación manual es fiel al forward original, en ambos modelos.")
```

```
[baseline] diferencia máxima (forward_with_head_ablated, sin ablar) vs forward
normal: 0.00e+00
[interpretable] diferencia máxima (forward_with_head_ablated, sin ablar) vs
forward normal: 0.00e+00
OK - la reimplementación manual es fiel al forward original, en ambos modelos.
```

1.5 Tarea 1 — Accuracy baseline por clase (sin ablación)

Forward pass completo sobre el sample de referencia, sin tocar nada. Se guarda el vector `accuracy_baseline_per_class` (dim 10) que es la referencia contra la que se mide el daño de cada cabeza.

[5]:
```
accuracy_baseline_global = {}
accuracy_baseline_per_class = {}
for key in MODELS:
    accuracy_baseline_global[key] = accuracy_of(direct_logits[key], ref_labels)
    accuracy_baseline_per_class[key] = per_class_accuracy(direct_logits[key],␣
 ↪ref_labels, num_classes)

    print(f"[{key}] Accuracy global baseline (n={N_REF}):␣
 ↪{accuracy_baseline_global[key]:.4f}")
    print(f"[{key}] Accuracy baseline por clase:")
    for c, name in enumerate(classes):
        print(f"  {name:>16}: {accuracy_baseline_per_class[key][c]:.4f}")
    print()
```

```
[baseline] Accuracy global baseline (n=1000): 0.2260
[baseline] Accuracy baseline por clase:
          antelope: 0.4785
      grizzly bear: nan
      killer whale: nan
            beaver: 0.0000
         dalmatian: nan
       persian cat: nan
             horse: nan
   german shepherd: nan
         blue whale: 0.0286
       siamese cat: nan
             skunk: nan
              mole: nan
             tiger: nan
      hippopotamus: nan
           leopard: nan
             moose: nan
     spider monkey: nan
    humpback whale: nan
          elephant: nan
           gorilla: nan
                ox: nan
               fox: nan
             sheep: nan
              seal: nan
        chimpanzee: 0.1027
           hamster: nan
```

```
          squirrel: nan
        rhinoceros: nan
            rabbit: nan
               bat: 0.0130
           giraffe: nan
              wolf: nan
         chihuahua: 0.3274
               rat: nan
            weasel: nan
             otter: nan
           buffalo: 0.3240
             zebra: nan
       giant panda: nan
              deer: nan
            bobcat: 0.0635
               pig: nan
              lion: nan
             mouse: nan
        polar bear: nan
            collie: 0.0789
            walrus: nan
           raccoon: nan
               cow: nan
           dolphin: nan

[interpretable] Accuracy global baseline (n=1000): 0.2300
[interpretable] Accuracy baseline por clase:
          antelope: 0.5646
      grizzly bear: nan
      killer whale: nan
            beaver: 0.0000
         dalmatian: nan
       persian cat: nan
             horse: nan
   german shepherd: nan
        blue whale: 0.0000
       siamese cat: nan
             skunk: nan
              mole: nan
             tiger: nan
      hippopotamus: nan
           leopard: nan
             moose: nan
     spider monkey: nan
    humpback whale: nan
          elephant: nan
           gorilla: nan
                ox: nan
```

```
          fox: nan
        sheep: nan
         seal: nan
   chimpanzee: 0.0753
      hamster: nan
     squirrel: nan
   rhinoceros: nan
       rabbit: nan
          bat: 0.0260
      giraffe: nan
         wolf: nan
    chihuahua: 0.0619
          rat: nan
       weasel: nan
        otter: nan
      buffalo: 0.2793
        zebra: nan
  giant panda: nan
         deer: nan
       bobcat: 0.2778
          pig: nan
         lion: nan
        mouse: nan
   polar bear: nan
       collie: 0.0921
       walrus: nan
      raccoon: nan
          cow: nan
      dolphin: nan
```

1.6 Tarea 2 — Vector de daño por clase, por cada cabeza del bloque

Para cada una de las `n_heads` cabezas de `block BLOCK_IDX`: se ablaciona **solo esa cabeza**, se corre el forward sobre el mismo sample, se calcula la accuracy por clase, y

```
damage_vector[head] = accuracy_baseline_per_class - accuracy_ablated_per_class   # dim 10
```

Daño positivo = esa clase empeora al apagar la cabeza (la cabeza ayudaba a esa clase). Daño negativo = la clase mejora al apagarla. Los `n_heads` vectores se apilan en `damage_matrix` de shape `(n_heads, 10)`.

[6]:
```
damage_matrix = {}
for key, (model, _, _) in models.items():
    damage_matrix[key] = np.zeros((n_heads, num_classes))
    for hidx in range(n_heads):
        logits = forward_with_head_ablated(model, ref_images,␣
 ↪ablate_layer_idx=block_idx0, ablate_head_idx=hidx)
        acc_c = per_class_accuracy(logits, ref_labels, num_classes)
```

```
        damage_matrix[key][hidx] = accuracy_baseline_per_class[key] - acc_c
        drop_global = accuracy_baseline_global[key] - accuracy_of(logits,␣
↪ref_labels)
        print(f"[{key}] block {BLOCK_IDX}, head {hidx + 1}: caída global =␣
↪{drop_global:+.4f}  |  "
              f"daño por clase [min {damage_matrix[key][hidx].min():+.3f}, max␣
↪{damage_matrix[key][hidx].max():+.3f}]")

    print(f"\n[{key}] damage_matrix (filas = cabezas, columnas = clases):")
    print(pd.DataFrame(damage_matrix[key], index=head_labels, columns=classes).
↪round(3))
    print()
```

```
[baseline] block 4, head 1: caída global = +0.0370  |  daño por clase [min +nan,
max +nan]
[baseline] block 4, head 2: caída global = +0.0150  |  daño por clase [min +nan,
max +nan]
[baseline] block 4, head 3: caída global = +0.0070  |  daño por clase [min +nan,
max +nan]
[baseline] block 4, head 4: caída global = +0.0310  |  daño por clase [min +nan,
max +nan]

[baseline] damage_matrix (filas = cabezas, columnas = clases):
        antelope  grizzly bear  killer whale  beaver  dalmatian  persian cat  \
head 1     0.105           NaN           NaN     0.0        NaN          NaN
head 2     0.033           NaN           NaN     0.0        NaN          NaN
head 3     0.019           NaN           NaN     0.0        NaN          NaN
head 4     0.072           NaN           NaN     0.0        NaN          NaN

        horse  german shepherd  blue whale  siamese cat  …  bobcat  pig  \
head 1    NaN              NaN       0.029          NaN  …   0.000  NaN
head 2    NaN              NaN       0.029          NaN  …   0.040  NaN
head 3    NaN              NaN       0.029          NaN  …   0.032  NaN
head 4    NaN              NaN       0.000          NaN  …   0.016  NaN

        lion  mouse  polar bear  collie  walrus  raccoon  cow  dolphin
head 1   NaN    NaN         NaN  -0.053     NaN      NaN  NaN      NaN
head 2   NaN    NaN         NaN   0.013     NaN      NaN  NaN      NaN
head 3   NaN    NaN         NaN   0.013     NaN      NaN  NaN      NaN
head 4   NaN    NaN         NaN   0.000     NaN      NaN  NaN      NaN

[4 rows x 50 columns]

[interpretable] block 4, head 1: caída global = +0.0060  |  daño por clase [min
+nan, max +nan]
[interpretable] block 4, head 2: caída global = +0.0050  |  daño por clase [min
+nan, max +nan]
```

```
[interpretable] block 4, head 3: caída global = -0.0070  |  daño por clase [min
+nan, max +nan]
[interpretable] block 4, head 4: caída global = +0.0080  |  daño por clase [min
+nan, max +nan]

[interpretable] damage_matrix (filas = cabezas, columnas = clases):
        antelope  grizzly bear  killer whale  beaver  dalmatian  persian cat  \
head 1    -0.053           NaN           NaN     0.0        NaN          NaN
head 2     0.053           NaN           NaN     0.0        NaN          NaN
head 3    -0.010           NaN           NaN     0.0        NaN          NaN
head 4     0.038           NaN           NaN     0.0        NaN          NaN

        horse  german shepherd  blue whale  siamese cat  …  bobcat  pig  \
head 1    NaN              NaN         0.0          NaN  …   0.063  NaN
head 2    NaN              NaN         0.0          NaN  …  -0.008  NaN
head 3    NaN              NaN         0.0          NaN  …  -0.032  NaN
head 4    NaN              NaN         0.0          NaN  …   0.008  NaN

        lion  mouse  polar bear  collie  walrus  raccoon  cow  dolphin
head 1   NaN    NaN         NaN  -0.026     NaN      NaN  NaN      NaN
head 2   NaN    NaN         NaN  -0.013     NaN      NaN  NaN      NaN
head 3   NaN    NaN         NaN  -0.013     NaN      NaN  NaN      NaN
head 4   NaN    NaN         NaN   0.000     NaN      NaN  NaN      NaN

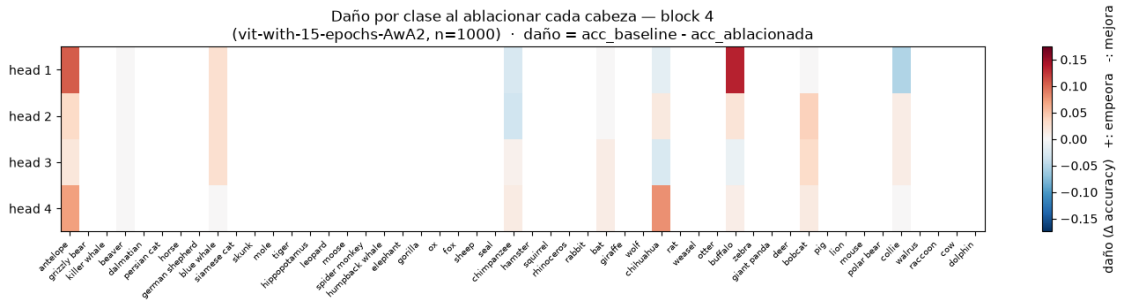
[4 rows x 50 columns]
```

[7]:
```
# --- Heatmap de la matriz de daño por clase (cabezas x clases) --------
# Escala divergente centrada en 0 (rojo = daña esa clase, azul = la mejora).␣
 ↪COMPARTIDA
# entre modelos: 30% arriba del máximo |daño| del baseline (sin penalización),␣
 ↪para que
# el mismo color sea comparable entre las dos figuras.
vmax_dmg = float(np.nanmax(np.abs(damage_matrix["baseline"]))) * 1.3
vmax_dmg = vmax_dmg if vmax_dmg > 0 else 1e-6

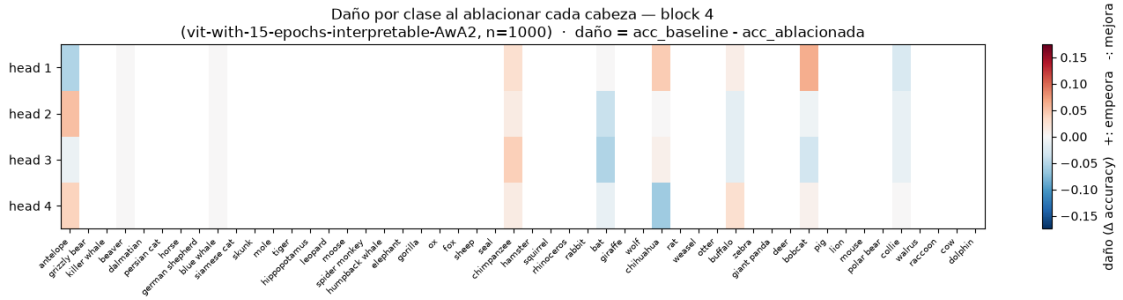
for key in MODELS:
    fig, ax = plt.subplots(figsize=(14, 3.6))
    im = ax.imshow(damage_matrix[key], cmap="RdBu_r", vmin=-vmax_dmg,␣
 ↪vmax=vmax_dmg, aspect="auto")
    ax.set_xticks(range(num_classes)); ax.set_xticklabels(classes, rotation=45,␣
 ↪ha="right", fontsize=7)
    ax.set_yticks(range(n_heads)); ax.set_yticklabels(head_labels)
    ax.set_title(f"Daño por clase al ablacionar cada cabeza - block␣
 ↪{BLOCK_IDX}\n"
                 f"({MODELS[key]}, n={N_REF})  ·  daño = acc_baseline -␣
 ↪acc_ablacionada")
```

```
    fig.colorbar(im, ax=ax, label="daño (Δ accuracy)   +: empeora   -: mejora")
    plt.tight_layout()
    save_fig(fig, "damage_by_class_heatmap", OUTPUT_DIR,␣
↪tag=f"{MODELS[key]}_block{BLOCK_IDX}", project_root=project_root)
    plt.show()
```

```
guardado: experimentation/experiments/30082026_head_specialization/damage_by_cla
ss_heatmap_vit-with-15-epochs-AwA2_block4.png
```




```
guardado: experimentation/experiments/30082026_head_specialization/damage_by_cla
ss_heatmap_vit-with-15-epochs-interpretable-AwA2_block4.png
```



1.7 Tarea 3 — Similitud pareada entre vectores de daño

Con `damage_matrix` (4×10), para los 6 pares `(1,2) (1,3) (1,4) (2,3) (2,4) (3,4)`:

- **Coseno**: `<d_i, d_j> / (||d_i|| ||d_j||)`. Sensible tanto a la *dirección* (qué clases) como al hecho de que ambos vectores apunten "hacia daño positivo".
- **Pearson** (control): coseno **después de centrar** cada vector (restarle su media). Al quitar la media, controla por el caso "esta cabeza daña más en general a todas las clases por igual" y deja solo el patrón *relativo* entre clases — si Pearson es bajo pero coseno alto, el parecido venía sobre todo de la magnitud media, no de atacar las mismas clases.

Se arman dos matrices `4×4` simétricas (diagonal = 1) y se reporta el **promedio de las 6 similitudes fuera de la diagonal** para cada métrica: un escalar de coseno y un escalar de Pearson por

corrida.

[8]:
```
from modules.similarity import offdiag_mean, pairwise_similarity_matrix


cosine_matrix, pearson_matrix = {}, {}
promedio_similitud_coseno, promedio_similitud_pearson = {}, {}

for key in MODELS:
    cosine_matrix[key]  = pairwise_similarity_matrix(damage_matrix[key],␣
 ↪"cosine")
    pearson_matrix[key] = pairwise_similarity_matrix(damage_matrix[key],␣
 ↪"pearson")

    promedio_similitud_coseno[key]  = offdiag_mean(cosine_matrix[key])
    promedio_similitud_pearson[key] = offdiag_mean(pearson_matrix[key])

    print(f"[{key}] Matriz de similitud COSENO (vectores de daño):")
    print(pd.DataFrame(cosine_matrix[key], index=head_labels,␣
 ↪columns=head_labels).round(3))
    print(f"[{key}] Matriz de similitud PEARSON (vectores de daño):")
    print(pd.DataFrame(pearson_matrix[key], index=head_labels,␣
 ↪columns=head_labels).round(3))
    print(f"[{key}] promedio_similitud_coseno  (6 pares) =␣
 ↪{promedio_similitud_coseno[key]:.4f}")
    print(f"[{key}] promedio_similitud_pearson (6 pares) =␣
 ↪{promedio_similitud_pearson[key]:.4f}\n")
```

```
[baseline] Matriz de similitud COSENO (vectores de daño):
        head 1  head 2  head 3  head 4
head 1     1.0     NaN     NaN     NaN
head 2     NaN     1.0     NaN     NaN
head 3     NaN     NaN     1.0     NaN
head 4     NaN     NaN     NaN     1.0
[baseline] Matriz de similitud PEARSON (vectores de daño):
        head 1  head 2  head 3  head 4
head 1     1.0     NaN     NaN     NaN
head 2     NaN     1.0     NaN     NaN
head 3     NaN     NaN     1.0     NaN
head 4     NaN     NaN     NaN     1.0
[baseline] promedio_similitud_coseno  (6 pares) = nan
[baseline] promedio_similitud_pearson (6 pares) = nan

[interpretable] Matriz de similitud COSENO (vectores de daño):
        head 1  head 2  head 3  head 4
head 1     1.0     NaN     NaN     NaN
head 2     NaN     1.0     NaN     NaN
head 3     NaN     NaN     1.0     NaN
```

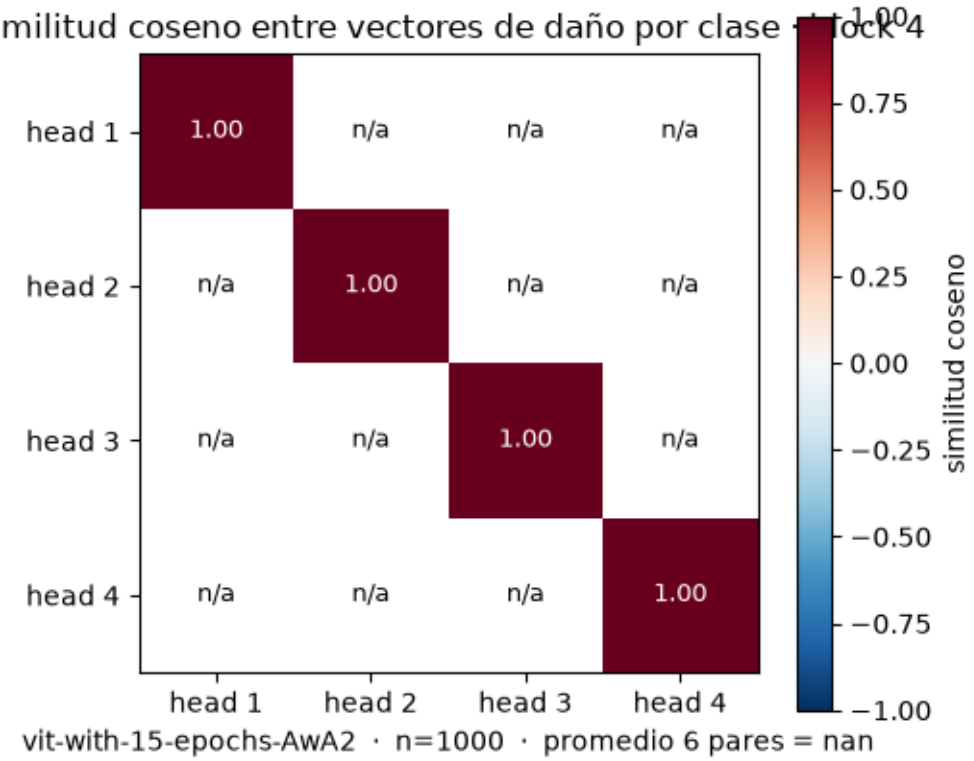
```
head 4     NaN     NaN     NaN     1.0
[interpretable] Matriz de similitud PEARSON (vectores de daño):
        head 1  head 2  head 3  head 4
head 1     1.0     NaN     NaN     NaN
head 2     NaN     1.0     NaN     NaN
head 3     NaN     NaN     1.0     NaN
head 4     NaN     NaN     NaN     1.0
[interpretable] promedio_similitud_coseno  (6 pares) = nan
[interpretable] promedio_similitud_pearson (6 pares) = nan


/Users/valefeve/Projects/interpretability of neural networks/interpretable-
transformers/modules/similarity.py:28: RuntimeWarning: Mean of empty slice
  return float(np.nanmean(M[iu]))
/Users/valefeve/Projects/interpretability of neural networks/interpretable-
transformers/modules/similarity.py:28: RuntimeWarning: Mean of empty slice
  return float(np.nanmean(M[iu]))
```

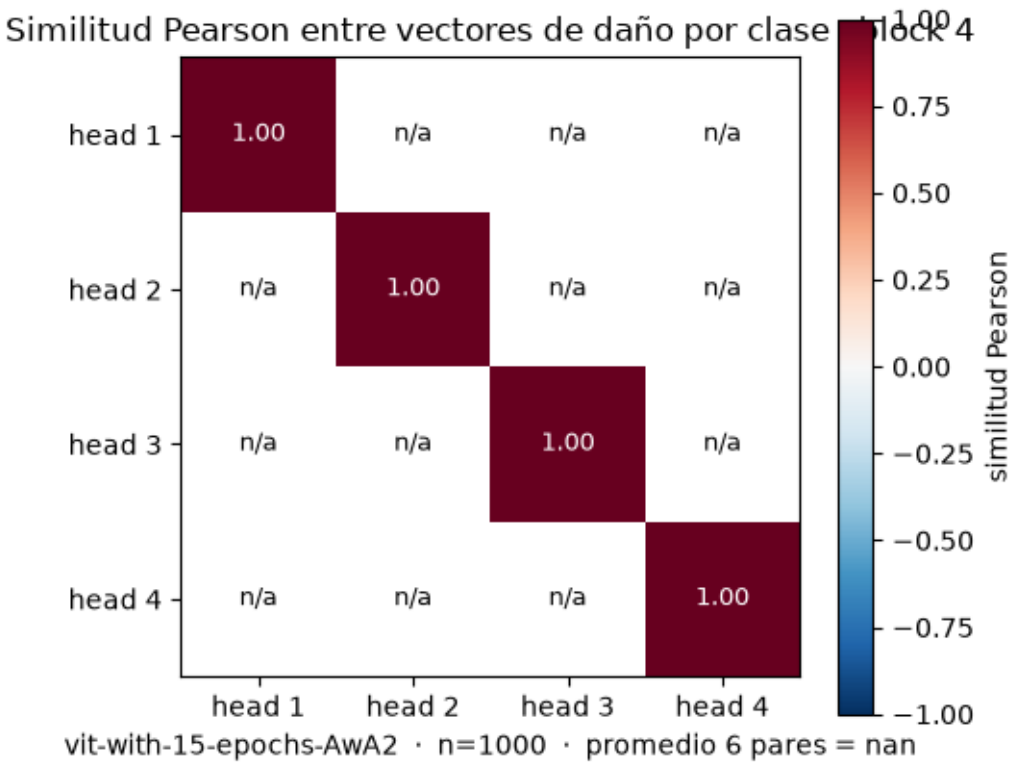
[9]:
```python
# --- Heatmaps de las dos matrices de similitud (4x4), un modelo por vez␣
 ↪-----------------
# Escala fija [-1, 1] para comparar entre corridas.
for key in MODELS:
    for M, name, stem in [
        (cosine_matrix[key],  "coseno",  "cosine_similarity_heatmap"),
        (pearson_matrix[key], "Pearson", "pearson_similarity_heatmap"),
    ]:
        fig, ax = plt.subplots(figsize=(5.2, 4.4))
        im = ax.imshow(M, cmap="RdBu_r", vmin=-1.0, vmax=1.0)
        ax.set_xticks(range(n_heads)); ax.set_xticklabels(head_labels)
        ax.set_yticks(range(n_heads)); ax.set_yticklabels(head_labels)
        for i in range(n_heads):
            for j in range(n_heads):
                v = M[i, j]
                ax.text(j, i, "n/a" if np.isnan(v) else f"{v:.2f}",␣
 ↪ha="center", va="center",
                        fontsize=9, color="white" if abs(np.nan_to_num(v)) > 0.
 ↪6 else "black")
        mean_val = promedio_similitud_coseno[key] if name == "coseno" else␣
 ↪promedio_similitud_pearson[key]
        ax.set_title(f"Similitud {name} entre vectores de daño por clase ·␣
 ↪block {BLOCK_IDX}")
        ax.set_xlabel(f"{MODELS[key]}  ·  n={N_REF}  ·  promedio 6 pares =␣
 ↪{mean_val:.3f}")
        fig.colorbar(im, ax=ax, label=f"similitud {name}")
        plt.tight_layout()
        save_fig(fig, stem, OUTPUT_DIR, tag=f"{MODELS[key]}_block{BLOCK_IDX}",␣
 ↪project_root=project_root)
```

```
        plt.show()
```

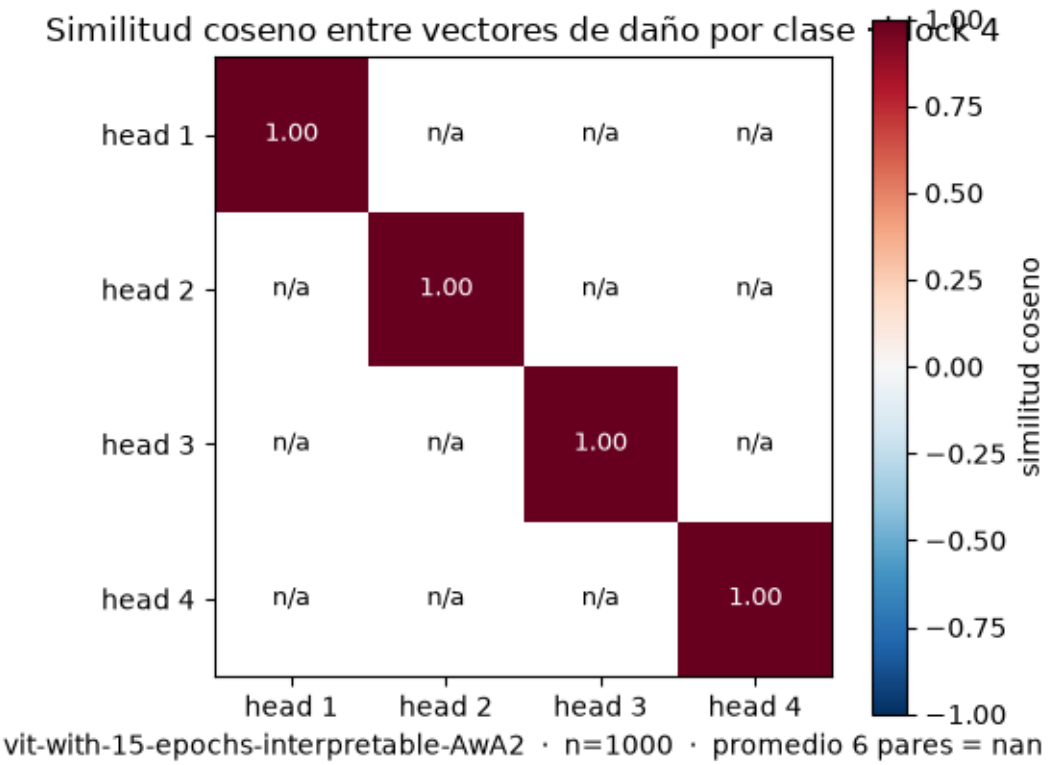
```
guardado: experimentation/experiments/30082026_head_specialization/cosine_simila
rity_heatmap_vit-with-15-epochs-AwA2_block4.png
```



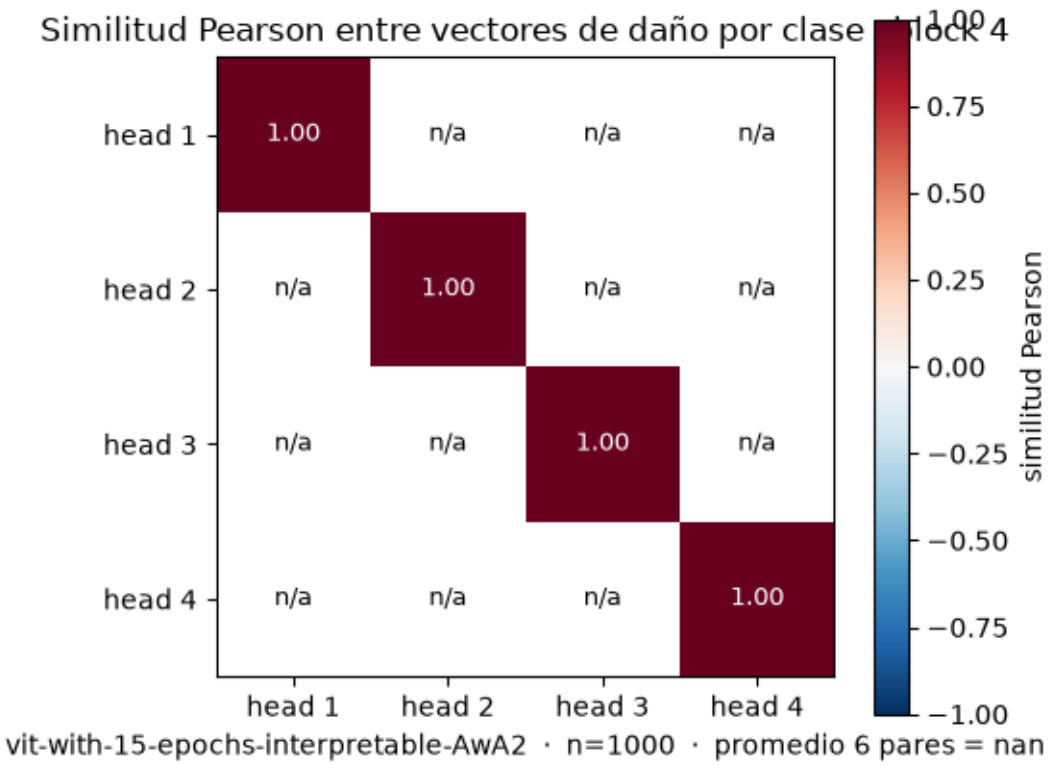

```
guardado: experimentation/experiments/30082026_head_specialization/pearson_simil
arity_heatmap_vit-with-15-epochs-AwA2_block4.png
```



guardado: experimentation/experiments/30082026_head_specialization/cosine_similarity_heatmap_vit-with-15-epochs-interpretable-AwA2_block4.png



guardado: experimentation/experiments/30082026_head_specialization/pearson_similarity_heatmap_vit-with-15-epochs-interpretable-AwA2_block4.png



1.8 Tabla resumen (CSV)

Un único CSV con todo lo del experimento: `accuracy_baseline_per_class` (10), `damage_vector` por cabeza (4×10), matriz coseno (4×4), matriz Pearson (4×4) y los dos escalares resumen. Cada fila lleva su etiqueta; las columnas de clase se usan para los vectores por clase, las de cabeza para las matrices 4×4, y la columna `scalar` para los dos promedios.

[10]:
```
rows_all = []
for key in MODELS:
    rows = {}
    rows["accuracy_baseline_per_class"] = {name:␣
 ↪accuracy_baseline_per_class[key][c] for c, name in enumerate(classes)}
    for h in range(n_heads):
        rows[f"damage_vector__{head_labels[h]}"] = {name: damage_matrix[key][h,␣
 ↪c] for c, name in enumerate(classes)}
    for h in range(n_heads):
        rows[f"cosine_similarity__{head_labels[h]}"] = {head_labels[j]:␣
 ↪cosine_matrix[key][h, j] for j in range(n_heads)}
    for h in range(n_heads):
        rows[f"pearson_similarity__{head_labels[h]}"] = {head_labels[j]:␣
 ↪pearson_matrix[key][h, j] for j in range(n_heads)}
```

```
    rows["promedio_similitud_coseno"]  = {"scalar":␣
↪promedio_similitud_coseno[key]}
    rows["promedio_similitud_pearson"] = {"scalar":␣
↪promedio_similitud_pearson[key]}

    df = pd.DataFrame.from_dict(rows, orient="index")
    df = df.reindex(columns=classes + head_labels + ["scalar"])
    df.index.name = "row"
    df.insert(0, "model_name", MODELS[key])
    df.insert(1, "variant", key)
    df.insert(2, "block_idx", BLOCK_IDX)
    rows_all.append(df)

summary_df = pd.concat(rows_all)
csv_path = OUTPUT_DIR / f"head_specialization_block{BLOCK_IDX}.csv"
summary_df.to_csv(csv_path)
print(f"guardado: {csv_path.relative_to(project_root)}")
summary_df
```

```
guardado: experimentation/experiments/30082026_head_specialization/head_speciali
zation_vit-with-15-epochs_block4.csv
```

[10]:

row	model_name	block_idx	plane	car	bird	cat	deer	dog
accuracy_baseline_per_class	vit-with-15-epochs	4	0.611650	0.629214	0.64	0.126214	0.222222	0.383721
damage_vector__head 1	vit-with-15-epochs	4	0.000000	-0.011236	0.02	-0.038835	0.055556	0.104651
damage_vector__head 2	vit-with-15-epochs	4	0.000000	0.067416	0.03	-0.029126	0.011111	0.034884
damage_vector__head 3	vit-with-15-epochs	4	0.019417	-0.022472	0.09	-0.009709	-0.044444	0.069767
damage_vector__head 4	vit-with-15-epochs	4	-0.029126	-0.022472	0.04	-0.048544	0.077778	-0.034884
cosine_similarity__head 1	vit-with-15-epochs	4	NaN					
cosine_similarity__head 2	vit-with-15-epochs	4	NaN					
cosine_similarity__head 3	vit-with-15-epochs	4	NaN					
cosine_similarity__head 4	vit-with-15-epochs	4	NaN					
pearson_similarity__head 1	vit-with-15-epochs	4	NaN					
pearson_similarity__head 2	vit-with-15-epochs	4	NaN					
pearson_similarity__head 3	vit-with-15-epochs	4	NaN					
pearson_similarity__head 4	vit-with-15-epochs	4	NaN					
promedio_similitud_coseno	vit-with-15-epochs	4	NaN					
promedio_similitud_pearson	vit-with-15-epochs	4	NaN					

cosine_similarity__head 1	NaN	NaN	NaN	NaN	NaN
cosine_similarity__head 2	NaN	NaN	NaN	NaN	NaN
cosine_similarity__head 3	NaN	NaN	NaN	NaN	NaN
cosine_similarity__head 4	NaN	NaN	NaN	NaN	NaN
pearson_similarity__head 1	NaN	NaN	NaN	NaN	NaN
pearson_similarity__head 2	NaN	NaN	NaN	NaN	NaN
pearson_similarity__head 3	NaN	NaN	NaN	NaN	NaN
pearson_similarity__head 4	NaN	NaN	NaN	NaN	NaN
promedio_similitud_coseno	NaN	NaN	NaN	NaN	NaN
promedio_similitud_pearson	NaN	NaN	NaN	NaN	NaN

row	frog	horse	ship	truck	head 1	\
accuracy_baseline_per_class	0.866071	0.823529	0.433962	0.651376	NaN	
damage_vector__head 1	0.026786	0.019608	-0.009434	0.018349	NaN	
damage_vector__head 2	-0.017857	0.019608	-0.028302	0.045872	NaN	
damage_vector__head 3	-0.008929	0.009804	-0.056604	0.045872	NaN	
damage_vector__head 4	0.133929	-0.039216	-0.009434	-0.009174	NaN	
cosine_similarity__head 1	NaN	NaN	NaN	NaN	1.000000	
cosine_similarity__head 2	NaN	NaN	NaN	NaN	0.444773	
cosine_similarity__head 3	NaN	NaN	NaN	NaN	0.442603	
cosine_similarity__head 4	NaN	NaN	NaN	NaN	0.266519	
pearson_similarity__head 1	NaN	NaN	NaN	NaN	1.000000	
pearson_similarity__head 2	NaN	NaN	NaN	NaN	0.326526	
pearson_similarity__head 3	NaN	NaN	NaN	NaN	0.402852	
pearson_similarity__head 4	NaN	NaN	NaN	NaN	0.247363	
promedio_similitud_coseno	NaN	NaN	NaN	NaN	NaN	
promedio_similitud_pearson	NaN	NaN	NaN	NaN	NaN	

row	head 2	head 3	head 4	scalar
accuracy_baseline_per_class	NaN	NaN	NaN	NaN
damage_vector__head 1	NaN	NaN	NaN	NaN
damage_vector__head 2	NaN	NaN	NaN	NaN
damage_vector__head 3	NaN	NaN	NaN	NaN
damage_vector__head 4	NaN	NaN	NaN	NaN
cosine_similarity__head 1	0.444773	0.442603	0.266519	NaN
cosine_similarity__head 2	1.000000	0.481286	-0.134928	NaN
cosine_similarity__head 3	0.481286	1.000000	-0.127443	NaN
cosine_similarity__head 4	-0.134928	-0.127443	1.000000	NaN
pearson_similarity__head 1	0.326526	0.402852	0.247363	NaN
pearson_similarity__head 2	1.000000	0.446750	-0.193198	NaN
pearson_similarity__head 3	0.446750	1.000000	-0.152108	NaN
pearson_similarity__head 4	-0.193198	-0.152108	1.000000	NaN
promedio_similitud_coseno	NaN	NaN	NaN	0.228802
promedio_similitud_pearson	NaN	NaN	NaN	0.179697

1.9 Resultado principal

[11]:
```
line = "=" * 68
print(line)
print(f"  RESULTADO PRINCIPAL  ·  block {BLOCK_IDX}  ·  n={N_REF}")
print(line)
for key in MODELS:
    print(f"  {key:>13} ({MODELS[key]})")
    print(f"      promedio_similitud_coseno   (6 pares de cabezas) =␣
 ↪{promedio_similitud_coseno[key]:+.4f}")
    print(f"      promedio_similitud_pearson  (6 pares de cabezas) =␣
 ↪{promedio_similitud_pearson[key]:+.4f}")
print(line)
print("  Más bajo = cabezas MÁS especializadas (vectores de daño por clase␣
 ↪menos")
print("  parecidos entre sí).")
print(line)
```

```
====================================================================
  RESULTADO PRINCIPAL  -  vit-with-15-epochs  ·  block 4  ·  n=1000
====================================================================
  promedio_similitud_coseno   (6 pares de cabezas) = +0.2288
  promedio_similitud_pearson  (6 pares de cabezas) = +0.1797
====================================================================
  Más bajo = cabezas MÁS especializadas (vectores de daño por clase menos
  parecidos entre sí). Comparar este par de números entre la corrida baseline
  y la interpretable, y entre bloques.
====================================================================
```

1.10 Para reflexionar / siguientes pasos

- Correr con `MODEL_NAME = "vit-with-15-epochs"` y con `"vit-with-15-epochs-interpretable"`, para el mismo `BLOCK_IDX`, y comparar `promedio_similitud_coseno` / `promedio_similitud_pearson`. La hipótesis del capítulo (B) es que el modelo penalizado los tiene **más bajos** (cabezas menos redundantes).
- Repetir barriendo `BLOCK_IDX` sobre los 4 bloques: ¿la especialización aparece sobre todo en los bloques finales (los que ya "deciden" la clase, cf. logit lens) o está repartida?
- El heatmap de daño por clase permite leer *qué* clases concentra cada cabeza: si en el modelo interpretable cada cabeza tiene su propio conjunto de clases "rojas" y en el baseline varias cabezas comparten las mismas, eso es evidencia cualitativa directa de especialización.
- Complemento natural (mismo sample): repetir con 2-3 semillas (**C.6**, *universality*) sobre estos dos escalares resumen.